

22. Синтаксическое дерево, даг. Атрибутная грамматика для построения дага. Проверка существования узла.

Вспомним схему устройства компилятора:

Frontend (занимается анализом, разбором, поиском ошибок)

Backend (занимается генерацией целевого кода).

Но между ними есть передаточное звено в виду промежуточного представления. После того, как блок анализа отработал, он получил некоторое представление исходного текста, пригодного для дальнейшей генерации кода на целевом языке, это и есть промежуточное представление, а **синтаксическое дерево** — один из его вариантов.

Для построения синтаксического дерева исходная грамматика должна быть операторной.

Синтаксическое дерево получается из дерева вывода применением двух преобразований, пока это возможно:

1. Листья, соответствующие терминалам-операторам (и ключевым словам), удаляются, а данные терминалы связываются с родительским узлом.
 2. Ветви дерева, соответствующие цепным правилам, сворачиваются в одну вершину.
(Важно учитывать, что не все листья в исходном дереве являются операторами и ключевыми словами. Только операторы идут в родительский узел, операнды остаются как есть, разве что поднимаются вверх цепочки, а сама цепочка при этом удаляется)
- В результате все узлы преобразованного дерева помечены терминалами (листья — операндами, а внутренние узлы — операторами и другими аналогичным по смыслу языковыми конструкциями)

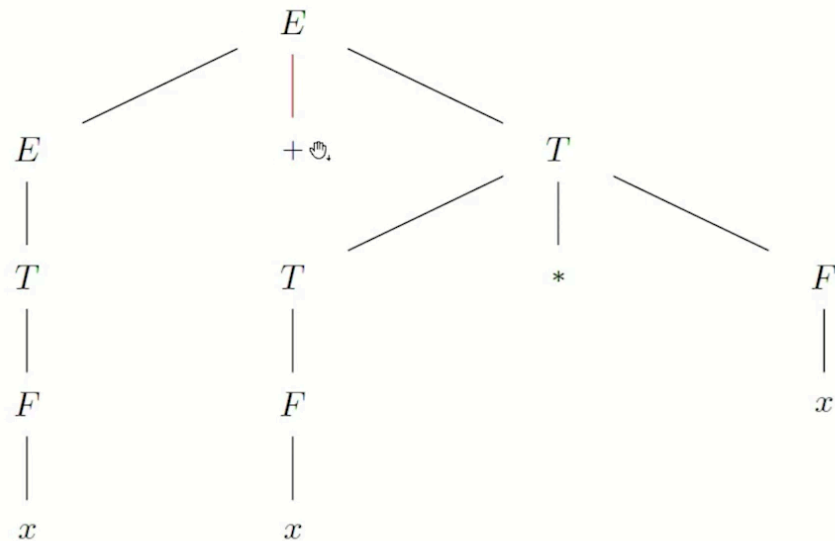
Если вдруг забыли, что такое дерево вывода:

Деревом вывода слова $w \in \Sigma^*$ в грамматике $G = (\Sigma, \Gamma, P, S)$ называется упорядоченное дерево, узлы которого помечены символами из $\Sigma \cup \Gamma \cup \{\lambda\}$ так, что:

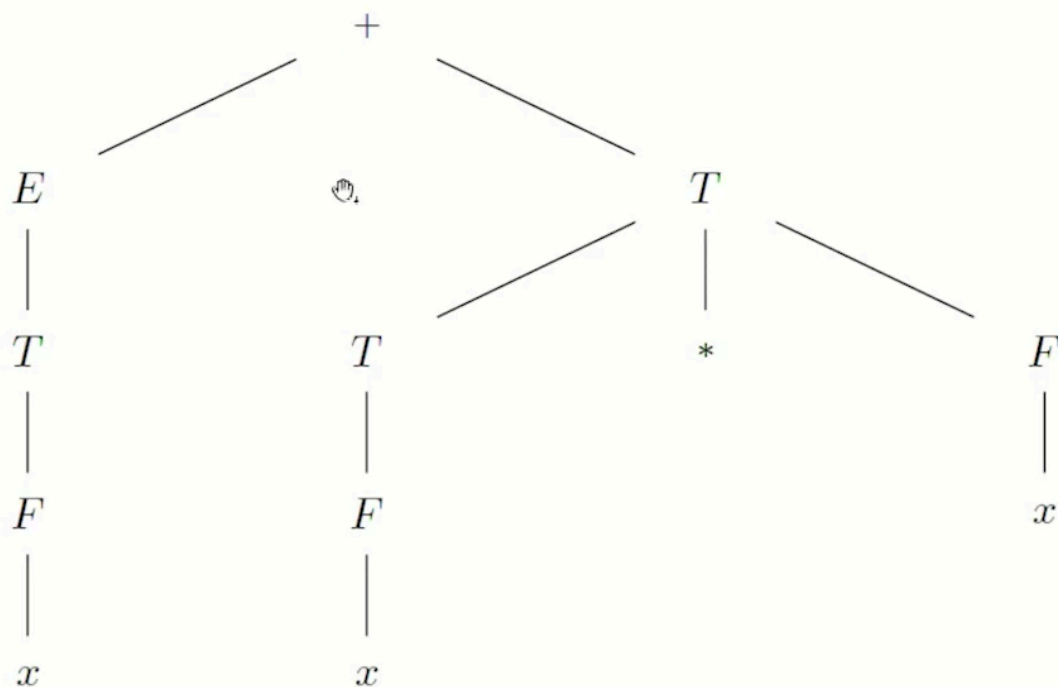
- 1) корень помечен аксиомой, внутренние узлы — нетерминалами, листья — терминалами или λ , причём узлы, помеченные λ , не имеют братьев;
- 2) если $y_1 \leq y_2 \leq \dots \leq y_k$ — все сыновья узла x в дереве, $Y_1, \dots, Y_k, X$ — их метки, то $(X \rightarrow Y_1 \dots Y_k) \in P$;
- 3) если $z_1 \leq z_2 \leq \dots \leq z_m$ — все листья дерева, $a_1, \dots, a_m$ — их метки, то $w = a_1 \dots a_m$.

Рассмотрим пример преобразования дерева вывода в синтаксическое дерево:

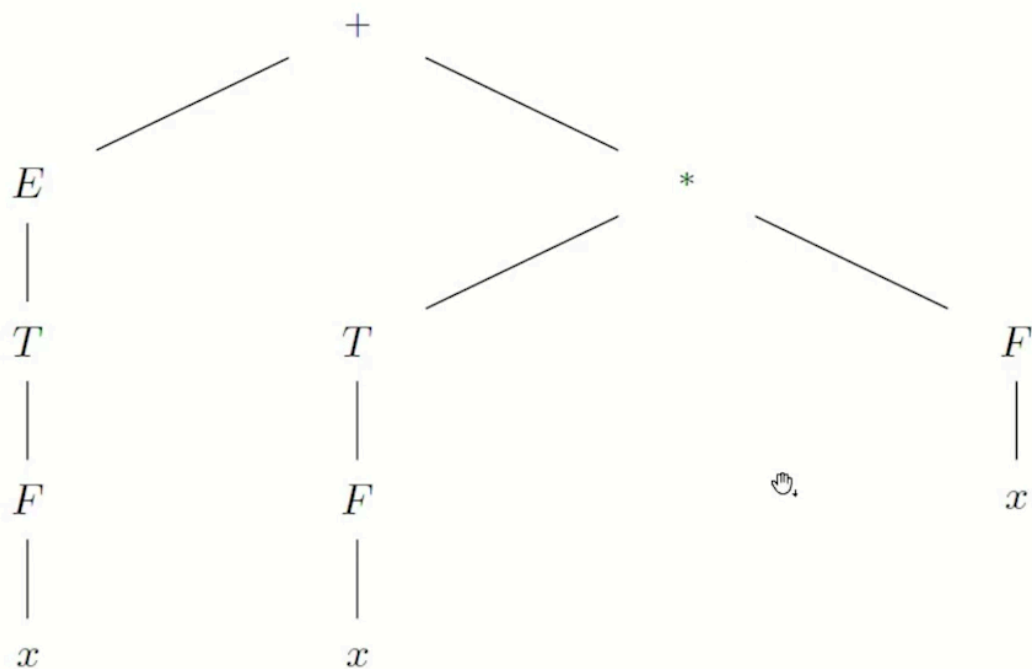
Преобразование дерева вывода в синтаксическое дерево на примере грамматики $E \rightarrow E + T | T, T \rightarrow T * F | F, F \rightarrow (E) | x$ и цепочки $x + x * x$.



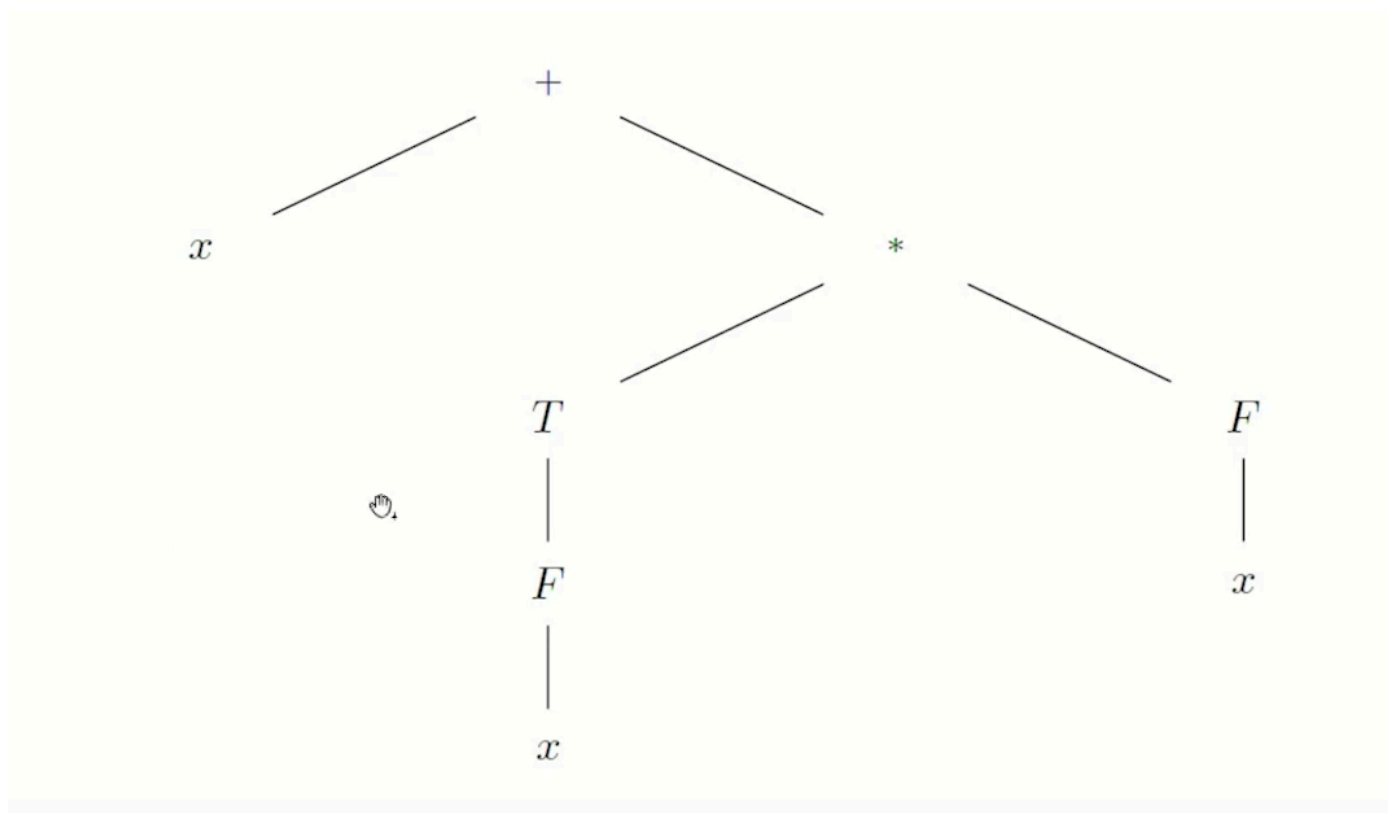
Поднимаем оператора в родительский узел: + Поднимаем в E.



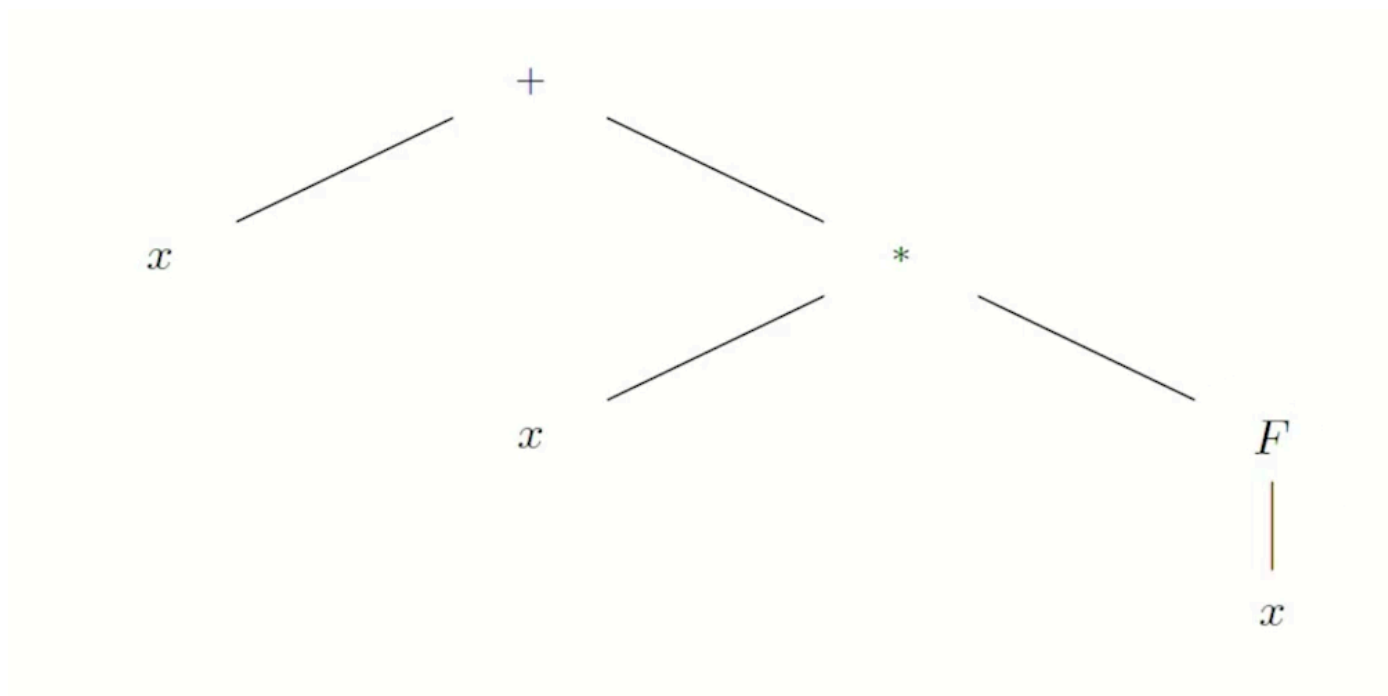
То же самое происходит с умножением в родительский узел T.



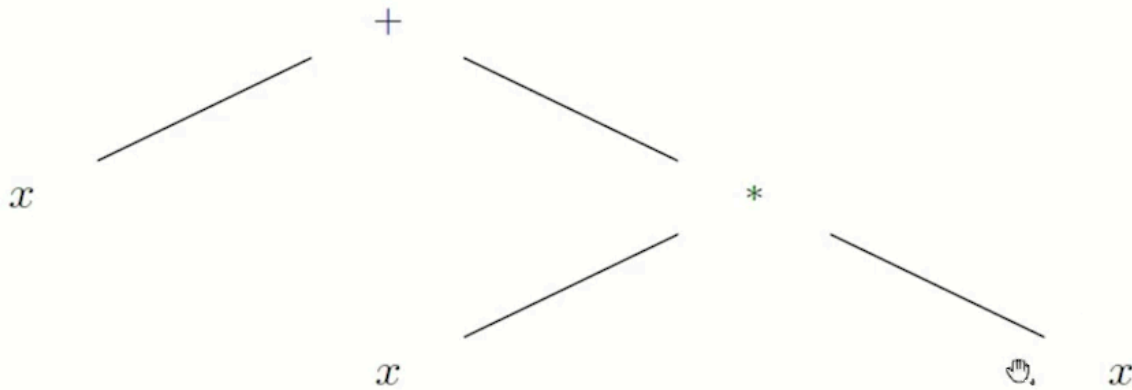
Теперь свернем цепочку E-T-F-x в один узел, помеченный соответствующим терминалом x



Аналогично T-F-x в x



Так же с F-x



Мы получили синтаксическое дерево!

Теперь рассмотрим, как выглядит атрибутная грамматика для синтаксического дерева:

Атрибутная грамматика для синтаксического дерева

Для КС-грамматики арифметических выражений построим атрибутную грамматику, которая будет строить синтаксическое дерево одновременно с выводом цепочки.

- Синтезируемый атрибут *ptr* у нетерминалов используется для хранения указателя на узел синтаксического дерева, соответствующий данному нетерминалу.
- Функция `MKLEAF(a, entry)` создает лист, помеченный операндом *a* и содержащий значение атрибута *entry* – указателя на запись в таблице символов.
- Функция `MKNODE(op, left, right)` создает узел операции, помеченный оператором *op* и содержащий указатели *left* и *right* на узлы операндов.

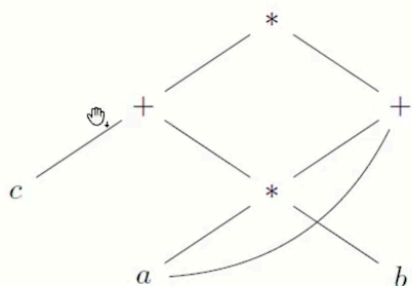
Атрибутная грамматика для синтаксического дерева

	Правило вывода	Семантические правила
1	$E \rightarrow E_1 + T$	$E.ptr = \text{MKNODE}(+, E_1.ptr, T.ptr)$
2	$E \rightarrow T$	$E.ptr = T.ptr$
3	$T \rightarrow T_1 * F$	$T.ptr = \text{MKNODE}(*, T_1.ptr, F.ptr)$
4	$T \rightarrow F$	$T.ptr = F.ptr$
5	$F \rightarrow (E)$	$F.ptr = E.ptr$
6	$F \rightarrow x$	$F.ptr = \text{MKLEAF}(x, x.entry)$

Что такое даг?

Синтаксический даг

Синтаксический даг (Directed Acyclic Graph) – это граф, получаемый из синтаксического дерева отождествлением его изоморфных поддеревьев. Даг так же, как дерево вывода и синтаксическое дерево, представляет синтаксическую структуру цепочки и имеет свойство упорядоченности: любое множество братьев линейно упорядочено «слева направо». При этом он гораздо компактнее. Кроме того, даг демонстрирует некоторые возможности оптимизации кода программы, которые могут быть использованы компилятором.



Представление
арифметического выражения
($c + a * b$) * ($a * b + a$) в виде дага

Какова будет атрибутная грамматика для построения дага? В функциях создании листа `MKLEAF()` и создании узла `MKNODE()` должны проверять, нет ли уже в строящемся даге изоморфного поддерева (например повторяющихся a^*b). Если такое поддерево уже есть, то просто вернем указатель на него и не будем создавать новый узел. Если нет, то делается всё, как и раньше — создается новый узел/лист.

Построение синтаксического дага

Плата за пространственную эффективность дага — временная сложность построения. Атрибутная грамматика для синтаксического дерева будет строить даг, если внести изменения в функции `MKLEAF()` и `MKNODE()`. Эти функции должны проверять существование идентичного узла; если такой узел есть, функция возвращает указатель на него, если идентичного узла нет — строит новый.

Процедура проверки существования узла с заданными параметрами может быть эффективно реализована с помощью хэша. Построенные узлы хранятся в виде массива списков.

Эффективный доступ к этой структуре данных обеспечивает хэш-функция, которая по произвольной тройке (*op*, *left*, *right*) однозначно (и быстро) вычисляет элемент массива заголовков. Далее просматривается список с данным заголовком на предмет наличия этого узла.

Основная загвоздка заключается в том, чтобы достаточно быстро проверять существование узла с заданными параметрами (т.е. с такой же меткой у корня и такими же левыми и правыми сыновьями или просто с такой же меткой, если речь о листе). Это можно реализовать например с помощью хэша, если хранить все построенные узлы в виде массива списков (все узлы разделяются на какие-то равные группы, и мы осуществляем с помощью хэша доступ к какой-то из этих групп и внутри этой группы её просматриваем на предмет наличия подходящего нам узла). Соответственно, хэш-функция по произвольной тройке (*op*, *left*, *right*) для `MKNODE` быстро вычисляет нужный заголовок, нужную ссылку на нужную группу узлов и далее этот список просматривается на предмет наличия интересующего нас узла, если такой узел найден, возвращаем на него ссылку, если узел не найден, то создаем новый узел. Таким образом мы можем минуя этап построения синтаксического дерева построить синтаксический даг.